

The Mixing Genetic Algorithm

Swetha Varadarajan
Department of Computer Science
Colorado State University
Fort Collins, CO-80521

March 1, 2020

Abstract

This research proposal is written for the Wim Bhom and Partners PhD 2020 award. The faculty advisor is Prof. Darrell Whitley. The proposed semester for funding is Fall 2020.

1 Introduction

The Traveling Salesman Problem (TSP) is one of the most studied NP-hard problems in the literature. Given a complete graph with N nodes, the goal of TSP is to find the shortest Hamiltonian circuit through the graph. TSP is used in applications such as vehicle routing, job sequencing, circuit design, and many more. In addition to these applications, TSP is often used in the design and analysis of heuristic methods [1]. Thus, it forms the subject of research interest in industry and academia.

The state-of-the-art TSP solvers fall under two broad categories. Concorde [1] is the best known exact TSP solver. It can solve problems of sizes up to 85,000 for optimality but with higher computational costs. Inexact solvers, on the other hand, can solve 100,000 size problems within a day or two. LKH [5] and EAX [9] are two well known inexact solvers. LKH is based on the Lin-Kernighan-Helsgaun algorithms and their variations. EAX, on the other hand, is based on the Genetic Algorithm (GA).

Recent studies [8] show that EAX with the restart mechanism performs the best on a wide variety of TSP instances. The EAX algorithm uses GAs, a class of algorithms inspired by Darwin's theory of evolution. These algorithms start with a population where every individual represents one possible solution in the search space. Operations such as selection, mutation, and crossover are applied to the population to produce the fittest individual. The operations on the individuals can occur in parallel and thereby expose tremendous parallelization opportunities.

MPI implementation [6] of the EAX algorithm exists in the literature. It can solve problems for sizes up to 200,000. However, there is no research to accelerate EAX on GPUs. With the decline of Moore's law, there is a need to shift the design of GAs from pure CPU-based to GPU and many-core architectures. Since the degree of parallelization of EAX is proportional to the population size, it can easily exploit the massively parallel architectures. The challenge, however, is in the prohibitively higher memory costs.

The EAX operator needs approximately $100 * N$ (N is the problem size) memory per crossover operation. EAX performance is also because of its highly optimized data structures for the CPUs. Significant efforts are required to create new data structures suitable for GPUs. On the other hand, Generalized Partitioned Crossover (GPX) is a deterministic crossover operator shown to improve EAX [10], LKH [4] and Concorde [11]. The stand-alone GPX consumes $4\times$ lesser memory than EAX. This key property makes **GPX an ideal candidate for GPU** implementation.

The ultimate goal of this research is to come up with new techniques to develop the best light-weight GA based TSP solver for GPU. We created the **Mixing Genetic Algorithm** (MGA) [12] using the GPX operator and show that it is one such promising technique. We also show that it is the **best implementation in the world**, wherein it can outperform the EAX in terms of the number of evaluations required to reach the global optima.

Related work: To the best of our knowledge, there are only three attempts [7, 3, 13] to implement GA based TSP solvers on GPUs. One notable work [7] is on a 100,000 city problem. However, this work reports results only on seven problem sizes, and they do not provide a detailed description of memory management. Two other GPU implementation [3, 13] for smaller problem sizes also exist. However, there is no information on the solution quality and the rate of convergence of these algorithms.

2 Research Status

The first key feature of MGA is its ability to avoid premature convergence. Traditionally, GPX recombines two parent strings, partitions them into q parts, and produces the best of the 2^q possible offspring solutions in $O(N)$ time complexity. However, there is a loss of half the number of edges for each crossover operation. In MGA, we use GPX to generate both the best and the worst offspring. This method of offspring generation preserves the population diversity and thereby prevent the algorithm from premature convergence.

The second property of MGA is its ability to pass all the information from one generation to the other. The information here is the edges that make up the TSP tour. By creating two complementary children (best and worst), MGA can pass all the edges from parent to offspring. Since no new edges are created, there are zero communication costs to the distance matrix. Thus, MGA is even more attractive and suitable for GPU implementation.

Table 1: Time line of the deliverables over the course of Fall 2020

Topic	Date	Targeted publication
MGA on MAXSAT	Aug 30	AAAI
Parallel implementations	Sept 30	IPDPS
MGA improvements	Oct 15	INFORMS Journal
Insights from parallelization	Nov 5	Parallel Computing Journal
Final dissertation	Dec 15	-

The third property of MGA is its ability to converge fast with increasing population size. This property also makes it an ideal operator for parallel machines. **MGA is the only TSP solver that has been executed on population size as large as 16384.** The above mentioned three properties have been explored on 25 TSP library instances on the CPU platform using OpenMP parallelization and published in GECCO 2019. The MGA algorithm is fast when compared to the EAX in terms of the number of recombinations. However, the execution time is much slower. Therefore, the algorithm has to be improved in three major aspects. One is the rate of convergence, second is the execution time, and the third is the scalability in terms of problem sizes.

The parallelization helps with the scalability and execution time issues. An MPI version of the EAX algorithm [6] can solve problems for sizes up to 200,000. MGA is also implemented to such large problem sizes using the MPI environment. The challenge is to modify the existing algorithm in such a way to make an appropriate trade-off between communication and computation costs. The questions to answer here are (1) Is there a real limit on the scalability of the MGA algorithm on the GPUs? (2) If yes, can an alternative data structure or algorithm be implemented to suit the GPU architectural needs to aid the scalability? (3) If not, what are the scalability limits with the MPI and GPU implementation? The results of this work are in progress, scheduled to completion by Summer 2020. The Fall semester will be focused on finishing up the writing.

The rate of convergence can be indirectly improved with parallelism. However, further research has to be made by examining the heuristic design as well. Some of the research questions to answer are, (1) Can intelligent

Table 2: Award Utilization

Expense type	amount
Living	\$7,750
Tuition fee (1 credit of thesis)	\$1,432.80
General fee	\$166.02
Alt. transportation fee	\$14.13
University Facility fee	\$20.75
University Technology fee	\$25.00
Health insurance	\$765
Health access fee	\$313.59
Parking fee	\$315
Gym	\$140
Conference travel	\$1,557.71
TOTAL	\$12,500

selection mechanisms help reduce the number of recombinations? (2) Can the GPX operator be further modified to execute fast? (3) Can the GPX operator be modified to produce intelligent offspring using multi-objective criteria? Will this accelerate the rate of convergence? To this end, we analyzed the progress of the MGA using visualization methods. Parameter, such as the multimodality and frequency distribution of the edges, gave further insights into the performance. Using this information, we were able to improve the rate of convergence of MGA. We were able to find out why certain TSP instances are much harder to solve on different TSP solvers. The paper has been submitted and is under review for GECCO 2020.

Furthermore, MGA can be applied to other combinatorial optimization problems such as MAX-SAT. GPX has been successfully used for MAX-SAT [2] on industrial applications. We have implemented a new lazier version of GPX that is theoretically shown to improve the execution time. The initial experimental results show promising results wherein we were able to solve some of the hard instances. Further research is in progress to compare the different mixing properties of MGA. The results of this work are in progress, scheduled to completion by Spring 2020. Summer and the early Fall semester will be focused on gathering the results and writing the thesis document on

the same. In Table 1, I provide the timeline of the deliverables.

The questions addressed in the all these studies will help understand more on the GA implementation costs on different architectures such as the single multi-core architecture (openMP), accelerator (GPU), and many-core architecture (MPI with openMP/GPU). These observations do not involve rigorous experimental setup. However, the details are tedious to obtain and are highly useful in the appropriate design of GA architectures and algorithms.

3 Conclusion

TSP is an important NP-hard problem studied extensively in the past century. The best solver for TSP is based on the inherently parallel genetic algorithms. The GPU implementation of these solvers is pertinent owing to the current advances in computer architectures. The proposed Mixing Genetic Algorithm has key properties that suit the GPU architectures.

First, the memory per crossover operation is much lesser when compared to the state-of-the-art CPU algorithms. Second, the communication cost for the function evaluations is zero. Third, the proposed algorithm has a higher rate of convergence in terms of the number of evaluations required to reach global optima. The current studies are aimed at improving the scalability, execution time, and the rate of convergence of MGA. The results of these studies are scheduled to be done by Summer 2020. The Fall semester will be focused on writing the results and preparing manuscripts for publication.

The primary reason for requesting the award is to spend on the expenditures shown in Table 2. Having not to worry about the necessary expenses, I will have the dedicated time required to produce high-quality writing. I will be able to provide valuable insights through an extensive literature search. Furthermore, this time will help produce manuscripts for the targeted publication venues shown in Table 1. I have a firm belief that this research will be useful to the CS department and the GA community.

Industrial experience: The motivation behind this research stems from an internship with a start-up company named Singular Computing. The requirement was to implement the GA based TSP solver on a massively parallel SIMD machine having 34,000 cores. The cores are arranged in a mesh architecture with minimal memory per core. These restrictions helped us understand the increasing demand for redesigning the well-established CPU based TSP solvers on the modern SIMD, massively parallel machines.

References

- [1] David L. Applegate, Robert E. Bixby, Vasek Chvatal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study (Princeton Series in Applied Mathematics)*. Princeton University Press, USA, 2007.
- [2] W. Chen, D. Whitley, R. Tinós, and F. Chicano. Tunneling between plateaus: improving on a state-of-the-art MAXSAT solver using partition crossover. In *Proc. of GECCO'2018*, pages 921–928, 2018.
- [3] Noriyuki Fujimoto and Shigeyoshi Tsutsui. A Highly-Parallel TSP Solver for a GPU Computing Platform. In Ivan Dimov, Stefka Dimova, and Natalia Kolkovska, editors, *Numerical Methods and Applications*, pages 264–271, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [4] D. Hains, D. Whitley, and A. Howe. Improving Lin-Kernighan-Helsgaun with crossover on clustered instances of the TSP. In *Proc. of PPSN XII*, pages 388–397. Springer, 2012.
- [5] K. Helsgaun. An effective implementation of the Lin-Kernighan traveling salesman heuristic. *European Journal of Oper. Res.*, 126(1):106–130, 2000.
- [6] K. Honda, Y. Nagata, and I. Ono. A parallel genetic algorithm with edge assembly crossover for 100,000-city scale TSPs. In *Proc. of the 2013 IEEE Congress on Evolutionary Computation*, pages 1278–1285, 2013.
- [7] Semin Kang, Sung-Soo Kim, Jongho Won, and Young-Min Kang. GPU-based parallel genetic approach to large-scale Travelling Salesman Problem. *The Journal of Supercomputing*, 72(11):4399–4414, Nov 2016.
- [8] Pascal Kerschke, Lars Kotthoff, Jakob Bossek, Holger H. Hoos, and Heike Trautmann. Leveraging tsp solver complementarity through machine learning. *Evolutionary Computation*, 26(4):597–620, 2018.
- [9] Y. Nagata and S. Kobayashi. A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. *INFORMS Journal on Computing*, 25(2):346–363, 2013.

- [10] D. Sanches, D. Whitley, and R. Tinós. Building a better heuristic for the traveling salesman problem: combining edge assembly crossover and partition crossover. In *Proc. of GECCO'2017*, 2017.
- [11] D. Sanches, D. Whitley, and R. Tinós. Improving an exact solver for the traveling salesman problem using partition crossover. In *Proc. of GECCO'2017*, 2017.
- [12] Swetha Varadarajan and Darrell Whitley. The massively parallel mixing genetic algorithm for the traveling salesman problem. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO 19, page 872879, New York, NY, USA, 2019. ACM.
- [13] Kai Zhang, Siman Yang, Li li, and Ming Qiu. Parallel Genetic Algorithm with OpenCL for Traveling Salesman Problem. In Linqiang Pan, Gheorghe Păun, Mario J. Pérez-Jiménez, and Tao Song, editors, *Bio-Inspired Computing - Theories and Applications*, pages 585–590, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.